Singly Linked List – Traverse List Inorder2 – Space Complexity and Algorithm.

Program:

```
void TraverseListInorder2(Node **head)
{
    if (*head == nullptr)
    {
        cout << "List is empty." << endl;
        return;
    }
    else
    {
  for (Node **current = head; *current != nullptr; current = &((*current)->next))
        {
            cout << (*current)->data << " ";
        }
        cout << endl;
    }
}

….
case 23:
        {
            cout << endl;
            cout << endl;
            cout << "Traverse List In Order 2:(For Loop)" << endl;
            TraverseListInorder2(&head);
            cout << endl;
            cout << endl;
            break;
        }
```

Space Complexity Analysis

1. *Input Space Complexity*

A. Parameter – Only Definition

The input space complexity refers to the space required for the input parameters of the function.

In this case, the function takes:

- ***A pointer to a pointer (Node ** head), which requires a constant amount of space, O(1).***

Thus, the total input space complexity is: O(1).

• Node **head – a single pointer-to-pointer (one machine word).

Fixed size ⇒ constant memory.

This view only considers the memory for the parameters passed to the function. The single parameter `Node **head` is a pointer-to-a-pointer, which has a **fixed, constant size**. Therefore, the input space complexity is **O(1)**.

B. Full – data (inclusive) Definition:

Count everything reachable from the parameter at the moment the call begins. *head may reference a singly linked list containing n nodes. Those n Node objects dominate, so the total input footprint is Θ(n).

This more common definition includes the entire data structure. If the linked list has **`n` nodes**, the space required to store that list is proportional to `n`. Therefore, the input space complexity is **Θ(n)**.

2. *Auxiliary Space Complexity*

Auxiliary space complexity refers to the extra space used by the algorithm that is not part of the input. In the case of the TraverseListInorder2 function, the auxiliary space is determined by the variables declared within the function:

- $\boldsymbol{The\ function\ uses\ a\ pointer\ Node\ *current\ to\ traverse\ the\ linked\ list, which\ requires\ O(1) space}$.

$\boldsymbol{Since\ the\ variable\ and\ process\ occupy\ a\ constant\ amount\ of\ space\ regardless\ of\ the\ size\ of\ the\ linked\ list, the\ auxiliary\ space\ complexity\ is}$: $\boldsymbol{O(1)}$.

What the routine itself adds beyond its input:

• Node **current – one local pointer-to-pointer.

• A few compiler-generated temporaries / return address.

• No dynamic heap allocations, no recursion, no additional data structures.

The amount of additional memory is constant, independent of n, therefore the auxiliary space complexity is O(1).

Auxiliary space is the extra memory an algorithm uses, not counting the input. For this function:

1. **`Node **current`**: The function declares only **one** extra variable, a pointer-to-a-pointer named `current`, to traverse the list. This variable occupies a constant amount of memory.
2. **Iteration**: The `for` loop reuses this single `current` pointer to move through the list. No new memory is allocated during the traversal.

Since the function only needs one extra pointer-to-a-pointer to do its job, regardless of the list's length, the amount of extra memory used is constant. The space used by the output stream (`cout`) is generally not counted as part of an algorithm's auxiliary space.

Therefore, the auxiliary space complexity is **Θ(1)**.

$Algorithm$

$TraverseListInorder2(START)$

$1.\ [START\ is\ NULL?\ ]if\ START = NULL, then:$

$Write: ``\ List\ is\ Empty.``$

$Return\ and\ Exit$

$Else$

$Repeat\ for\ \ CURRPTR = START\ to\ CURRPTR \neq NULL, DO:$

$Write: INFO[CURRPTR]$

$UPDATE\ AND\ \ SET\ \ CURRPTR := ADDRESS\ of\ LINK[CURRPTR]$

$[END\ of\ LOOP]$

$[END\ of\ IF\ structure]$

$[END\ of\ Algorithm\ ]$

$[Alternative\ ]$

$TraverseListInorder2(START)$

$1.\ [START\ is\ NULL?\ ]if\ START = NULL, then:$

$Write: ``\ List\ is\ Empty.``$

$Return\ and\ Exit$

$Else$

$FOR(SET\ \ CURRPTR := START\ ;\ CURRPTR \neq NULL\ ;\ SET\ \ CURRPTR := ADDRESS\ of\ LINK[CURRPTR])DO\ :$

$Write: INFO[CURRPTR]$

$[END\ of\ LOOP]$

$[END\ of\ IF\ structure]$

$[END\ of\ Algorithm\ ]$